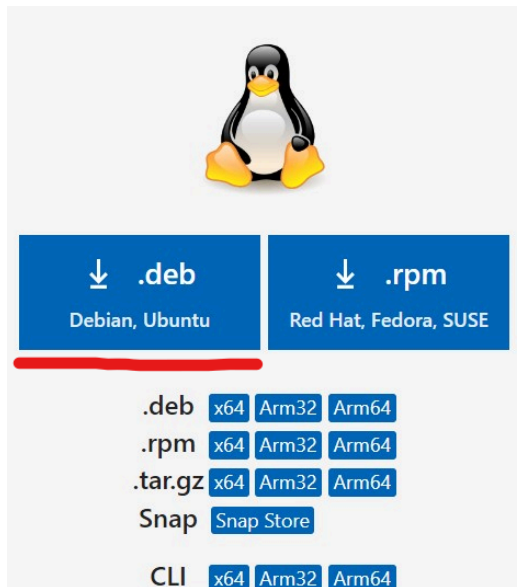
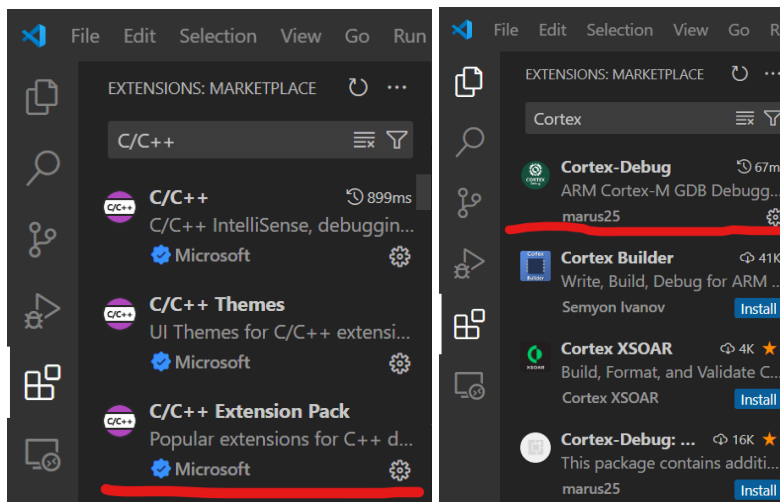


This guide is **ONLY APPLICABLE FOR LINUX** (Ubuntu example).
Use this guide for your own machine (either Linux or WSL for Windows).
Class computers have everything pre-installed.

1. Install Visual Studio Code (VS Code) <https://code.visualstudio.com/>. OR
VSCodium <https://vscodium.com/>.



2. Install the “C/C++ Expansion Pack” and “Cortex Debug” extensions in VS Code.



(Press the little 'Install' blue buttons on the right bottom side of C/C++ Extension Pack and Cortex-Debug. In this example they are already installed.)

3. Install OpenOCD (linux-x64)
<https://github.com/xpack-dev-tools/openocd-xpack/releases/>:

xpack-openocd-0.12.0-3-darwin-arm64.tar.gz	2.14 MB	Apr 2
xpack-openocd-0.12.0-3-darwin-arm64.tar.gz.sha	109 Bytes	Apr 2
xpack-openocd-0.12.0-3-darwin-x64.tar.gz	2.22 MB	Apr 2
xpack-openocd-0.12.0-3-darwin-x64.tar.gz.sha	107 Bytes	Apr 2
xpack-openocd-0.12.0-3-linux-arm.tar.gz	2.36 MB	Apr 2
xpack-openocd-0.12.0-3-linux-arm.tar.gz.sha	106 Bytes	Apr 2
xpack-openocd-0.12.0-3-linux-arm64.tar.gz	2.44 MB	Apr 2
xpack-openocd-0.12.0-3-linux-arm64.tar.gz.sha	108 Bytes	Apr 2
<u>xpack-openocd-0.12.0-3-linux-x64.tar.gz</u>	2.49 MB	Apr 2
xpack-openocd-0.12.0-3-linux-x64.tar.gz.sha	106 Bytes	Apr 2

Installation of OpenOCD latest version at the time of writing this guide.

Unpack the package into `/usr/share/`:

```
sudo tar -xf openocdpackage_location -C /usr/share
```

Add a symbolic link:

```
sudo ln -s /usr/share/xpack-openocd-your_version/bin/openocd /usr/bin/openocd
```

4. Install the “`60-openocd.rules`” file for device recognition. Navigate to the unpacked location.

```
cd /usr/share/xpack-openocd-your_version/openocd/contrib
```

Move the file to `/etc/udev/rules.d/`:

```
sudo mv 60-openocd.rules /etc/udev/rules.d/
```

Apply changes:

```
sudo udevadm control --reload
```

If “*Failed to send reload request: No such file or directory*”, then run:
`sudo service udev restart`, and try `sudo udevadm control --reload` again.

Then:

```
sudo udevadm trigger
```

IF YOU EVER RUN INTO TROUBLES WHEN USING OPENOCD (MOST LIKELY ON WSL, LIKE: `Error: libusb open() failed with LIBUSB_ERROR_ACCESS`), RUN THESE COMMANDS IN THIS ORDER (OR USE THE `wsl_udev.sh` SCRIPT FROM LAB CATALOG TO RUN THESE COMMANDS):

```
sudo service udev restart
sudo udevadm control --reload
sudo udevadm trigger
```

After that, plug in the LaunchPad board and confirm that **Luminary Micro Inc. In-Circuit Debug Interface** is recognised. To confirm run in the terminal:

```
lsusb
```

Output:

```
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 002: ID 1cbe:00fd Luminary Micro Inc. In-Circuit Debug Interface
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

If you are using WSL, then to connect the Launchpad, install *usbipd* following instructions from Microsoft:

<https://learn.microsoft.com/en-us/windows/wsl/connect-usb>

Confirm that OpenOCD works properly (make sure that the LaunchPad is connected!):

```
openocd -f board/ti_ek-tm4c123gxl.cfg
```

Output:

```
Info : The selected transport took over low-level target control. The results might differ compared to plain JTAG/SWD
Info : Listening on port 6666 for tcl connections
Info : Listening on port 4444 for telnet connections
Info : clock speed 21846 kHz
Info : ICDI Firmware version: 9270
Error: SRST error
Info : tm4c123gh6pm.cpu: hardware has 6 breakpoints, 4 watchpoints
Info : starting gdb server for tm4c123gh6pm.cpu on 3333
Info : Listening on port 3333 for gdb connections
```

Correctly working OpenOCD with listening on the port 3333

Press **Ctrl + C** to terminate the process if successful.

5. Install ARM GNU Toolchain for debugging and code compilation (search for **x86_64 Linux hosted cross toolchains (arm-none-eabi)**)

<https://developer.arm.com/downloads/-/arm-gnu-toolchain-downloads>

x86_64 Linux hosted cross toolchains

AArch32 bare-metal target (arm-none-eabi)

- [arm-gnu-toolchain-13.2.rel1-x86_64-arm-none-eabi.tar.xz](#)
- [arm-gnu-toolchain-13.2.rel1-x86_64-arm-none-eabi.tar.xz.asc](#)
- [arm-gnu-toolchain-13.2.rel1-x86_64-arm-none-eabi.tar.xz.sha256asc](#)

Installation of arm-gnu-toolchain version at the time of writing this guide.

Unpack the package into `/usr/share/`:

```
sudo tar -xf arm-gnu-toolchain_your_location -C /usr/share/
```

Add symlinks using these commands:

```
sudo ln -s /usr/share/arm-gnu-toolchain-your_version/bin/* /usr/bin/
```

Or manually each thing inside bin (arm-none-eabi-gcc, arm-none-eabi-gdb, etc):

```
sudo ln -s /usr/share/arm-gnu-toolchain-your_version/bin/arm-none-eabi-X /usr/bin/arm-none-eabi-X
```

Confirm that everything is working:

```
arm-none-eabi-gcc --version
```

```
arm-none-eabi-gdb (if works type quit)
```

NEXT STEPS MAY BE NEEDED TO BE PERFORMED IN CASE OF SPECIFIC ERRORS!

Symlink dependencies errors: *shared libraries: libncursesw.so.5: cannot open shared object file: No such file or directory*. To fix type in terminal:

```
sudo ln -s /usr/lib/x86_64-linux-gnu/libncursesw.so.6 /usr/lib/x86_64-linux-gnu/libncursesw.so.5
```

Python 3.8. PYTHONHOME and PYTHONPATH not set, ModuleNotFoundError: No module named 'encodings'. To fix install python 3.8, type in terminal:

```
sudo add-apt-repository ppa:deadsnakes/ppa
```

```
sudo apt update
```

```
sudo apt install python3.8
```

6. **Install TivaWare Software Development Kit**
<https://www.ti.com/tool/download/SW-TM4C>

Choose the 2013 2.0 version that doesn't require registration and approval.

TivaWare for C Series Software (Complete)

Select a version

Version: 2.0 Release date: 27 Aug 2013

Filter by version or date



v2x

2.2.0.295 (22 Apr 2020)

2.1.4.178 (21 Feb 2017)

2.1.3.156 (24 Jul 2016)

2.1.2.111 (15 Dec 2015)

2.1.1.71 (06 May 2015)

2.0.1.11577 (07 Oct 2013)

2.0 (27 Aug 2013)

v1x

1.1 (30 Jun 2013)

[Release notes](#)

[View software details](#)

Downloads

[Supported products & hardware](#)



SW-TM4C-2.0.exe — 152328 K



SW-TM4C-DRL-2.0.exe — 7732 K



SW-TM4C-GRL-2.0.exe — 7821 K

Scroll down and install the .exe file.

[SW-EK-TM4C123GXL-2.0.exe](#) — 33936 K

EK-TM4C123GXL Kit Software

MD5 checksum

de01a1a34ab3d89c5c5e67a6ebc0ce78



Create a new workspace folder (If you are using the class computer create in the P drive):

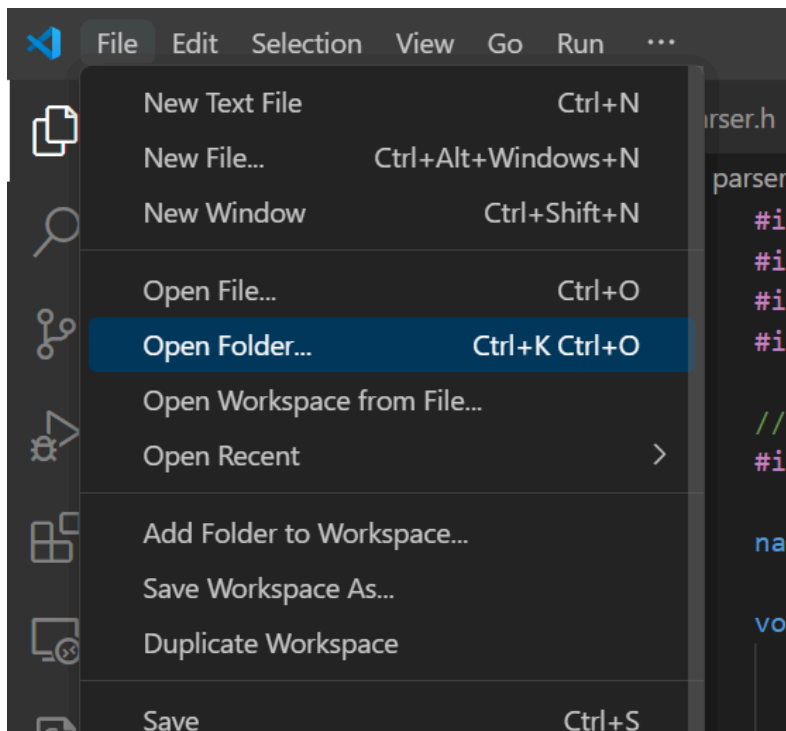
```
mkdir PATH_TO_WORKSPACE/tivaware
```

Move the .exe file to this folder and unpack it (this command works only if the .exe is in the workspace folder):

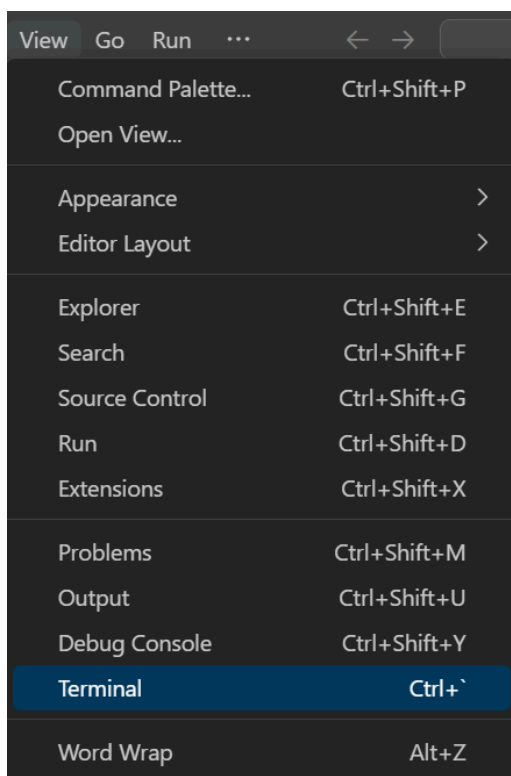
```
unzip FILENAME.exe
```

7. Download the BESLabs catalog into `PATH_TO_WORKSPACE/tivaware/examples/`
<https://gitlab.cs.ttu.ee/ajasts/beslabs>
8. Open the folder
`PATH_TO_WORKSPACE/tivaware/examples/BESLabs_catalog/lab1`
in VS Code:

Choose *File -> Open folder...* and open the specified folder.



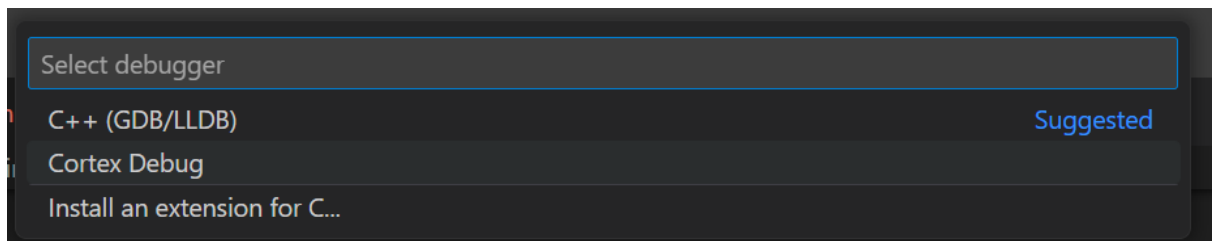
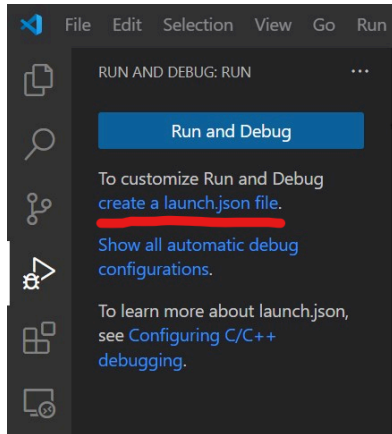
9. **Compile and build the .axf file with debug info (open Terminal in VS Code with View -> Terminal):**



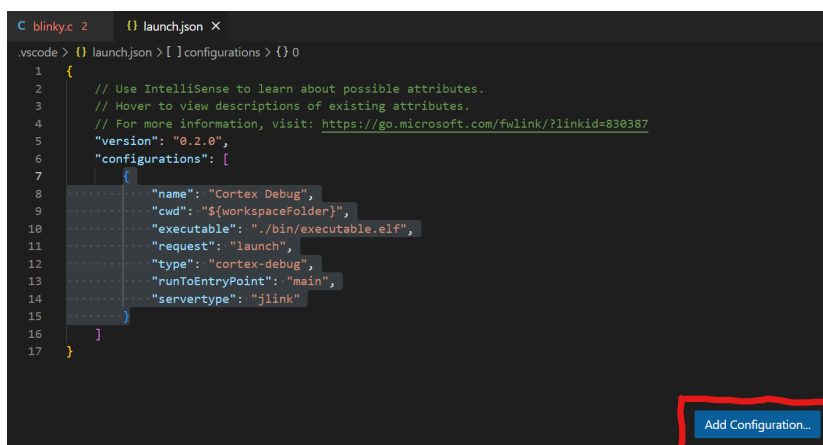
And type in `make`.

10. Open the sidebar panel “Run and Debug” and configure the file for Cortex Debug:

Press “create a launch.json file” and then choose *Cortex Debug*.



Delete the highlighted configuration and press “Add configuration”, choose *Cortex Debug: OpenOCD*.



```
vscode > [ ] launch.json > [ ] configurations
1 {
2     // Use IntelliSense to learn about possible attributes.
3     // Hover to view descriptions of existing attributes.
4     // For more information, visit: https://go.microsoft.com/fwlink/?linkid=829397
5     "version": "0.2.0",
6     "configurations": [
7         {} C/C++: (gdb) Pipe Launch
8         {} Chrome: Attach
9         {} Chrome: Launch
10        {} CMake: Configure
        {} CMake: External
        {} CMake: Script
        {} Cortex Debug: JLink
        {} Cortex Debug: OpenOCD
        {} Cortex Debug: PyOCD
        {} Cortex Debug: ST-LINK
        {} Cortex Debug: ST-Util
        {} Edge: Attach
        {} Edge: Launch
    ]
}
```

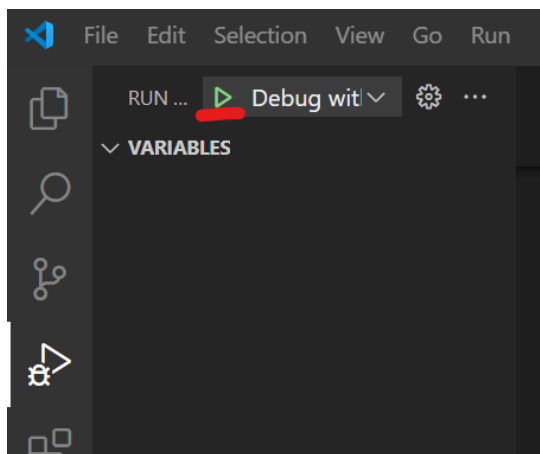
After that change the next lines to these values (copying these lines might produce errors, so type them out!):

```
"executable": "./gcc/main.axf"
```

```
"configFiles": ["board/ti_ek-tm4c123gxl.cfg"]
```

You have to create a new debugging configuration for each separate lab.

11. Run the debugger (it will automatically run the OpenOCD and the gdb and then flash the LaunchPad with the .axf file):



IN CASE OF CODE UPDATE, REMEMBER TO FIRST COMPILE AND BUILD (SEE STEP 9) AND ONLY AFTER DEBUG SO THAT THE NEW CODE GETS FLASHED ON THE BOARD!

12. If everything was set up correctly and no errors encountered, it is possible to see Global and Local variables, Step in code, continue or Stop, set Breakpoints, etc (the debugger always starts at `main()`).

**Sometimes it might be needed to press the RESET button on the LaunchPad!
(before or after debugging/flashng)**

13. If you do not wish to use the debugger and only want to build and flash the board, use these commands in the terminal:

```
make
```

```
../flash.sh
```